

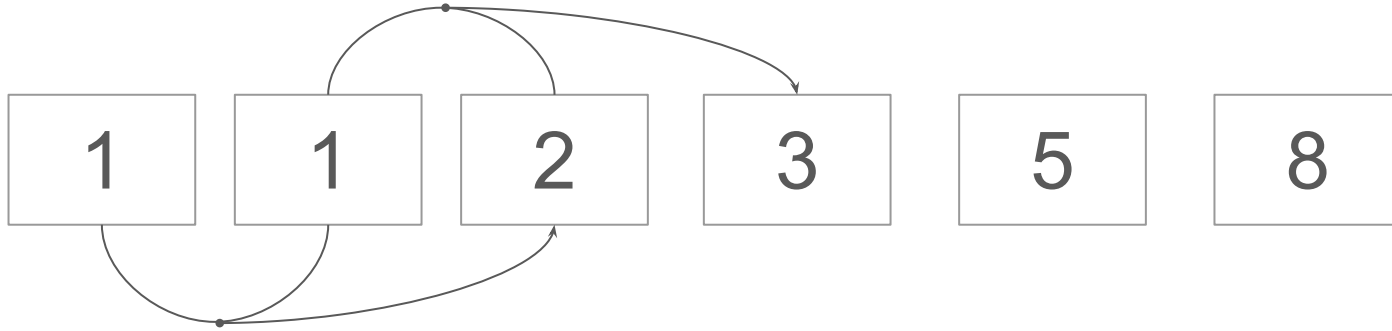
Dynamic Programming

Algorithms for solving recursive problems

What are recursive problems?

Involves smaller but similar repeated sub-problems.

Example: Fibonacci Numbers (Sum of previous two terms)



The Programming Solution (Python)

The problem of recursion

We are repeating computations!

The number of operations double for every unit increase in N.

We avoid repetition by **saving solutions to pre-computed functions**.

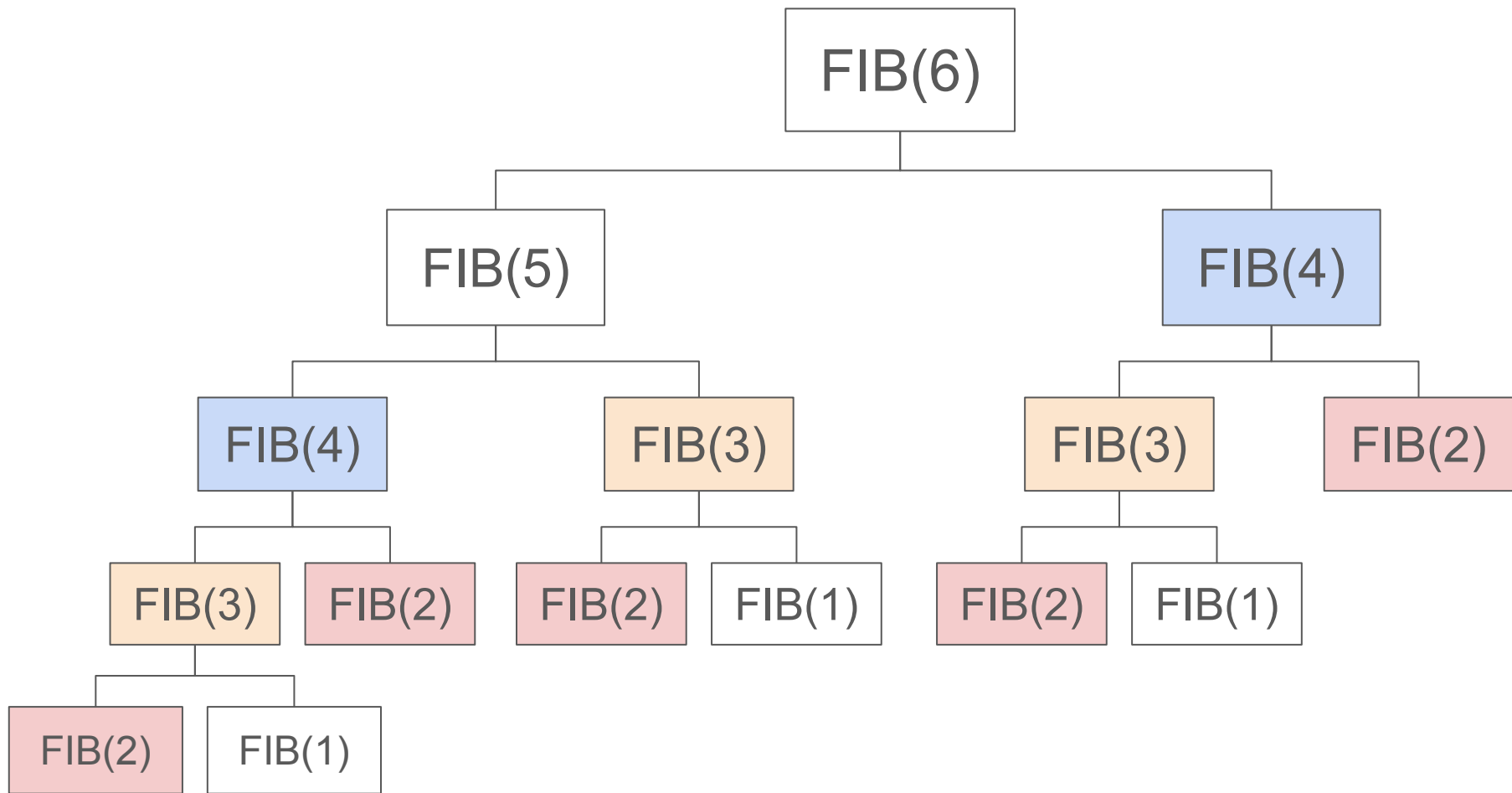


1,048,576

20

1.27×10^{30}

100

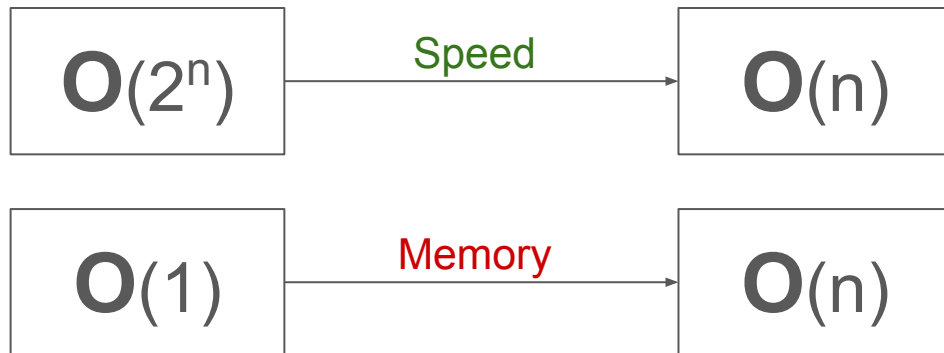


Adding Memoization in Python Code

The Problem of Space/Memory

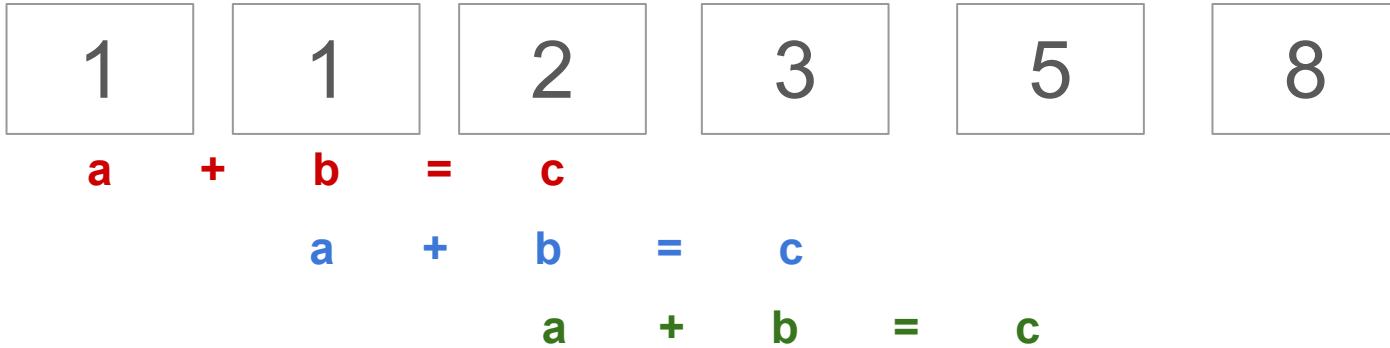
We've been working with a **top-down approach** with memoization added to avoid recomputation of solved functions. Requires a high space usage.

Like computational power, **memory is also a limited resource**.



The bottom-up approach

Maintain the time complexity while reducing the space complexity.



Bottom-up over top-down approach

The bottom-up approach is best dynamic programming approach. It will always **maintain the time complexity** without massively increasing the space complexity.

	Recursion	Top-Down DP	Bottom-up DP
Speed	$O(2^n)$	$O(n)$	$O(n)$
Memory	$O(1)$	$O(n)$	$O(1)$

Recap

1. In dynamic programming, we solve the main problem by **finding solutions to the smaller repeated problems**.
2. There are **two approaches** to solve dynamic programming problems
3. The **top-down approach** where we create a recursive function
4. We use **memoization** to avoid resolving already called functions
5. The **bottom-up approach** is the most ideal approach as it avoids resolving called functions while still optimising the space complexity

Example: House Robber Challenge

House Robber Leetcode Challenge

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed.

Adjacent houses **have security systems connected** and it will automatically contact the police if two adjacent houses were broken into on the same night.

Return the **maximum amount of money you can rob** tonight without alerting the police.

Example Test Cases

→ Input: nums = [1,2,3,1]

Output: 4

→ Input: nums = [2,7,9,3,1]

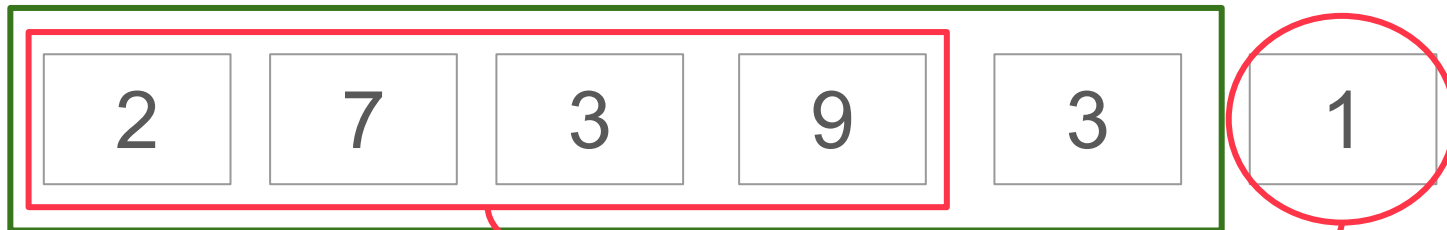
Output: 12

→ Input: nums = [2,1,1,2]

Output: 4

Top-Down Dynamic Programming Approach

Option B



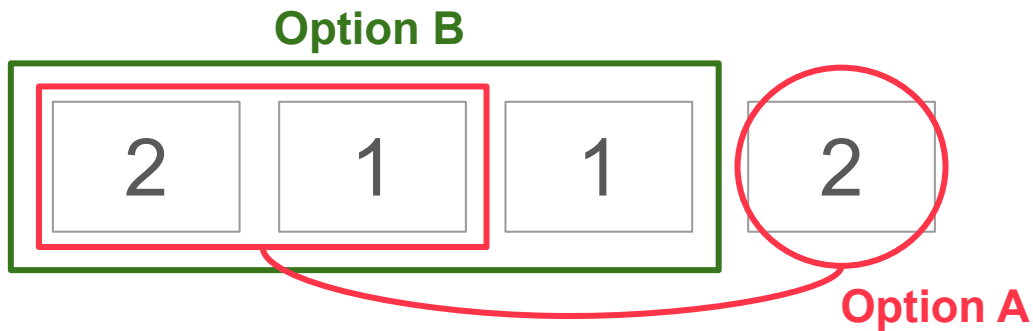
Option A

Option B

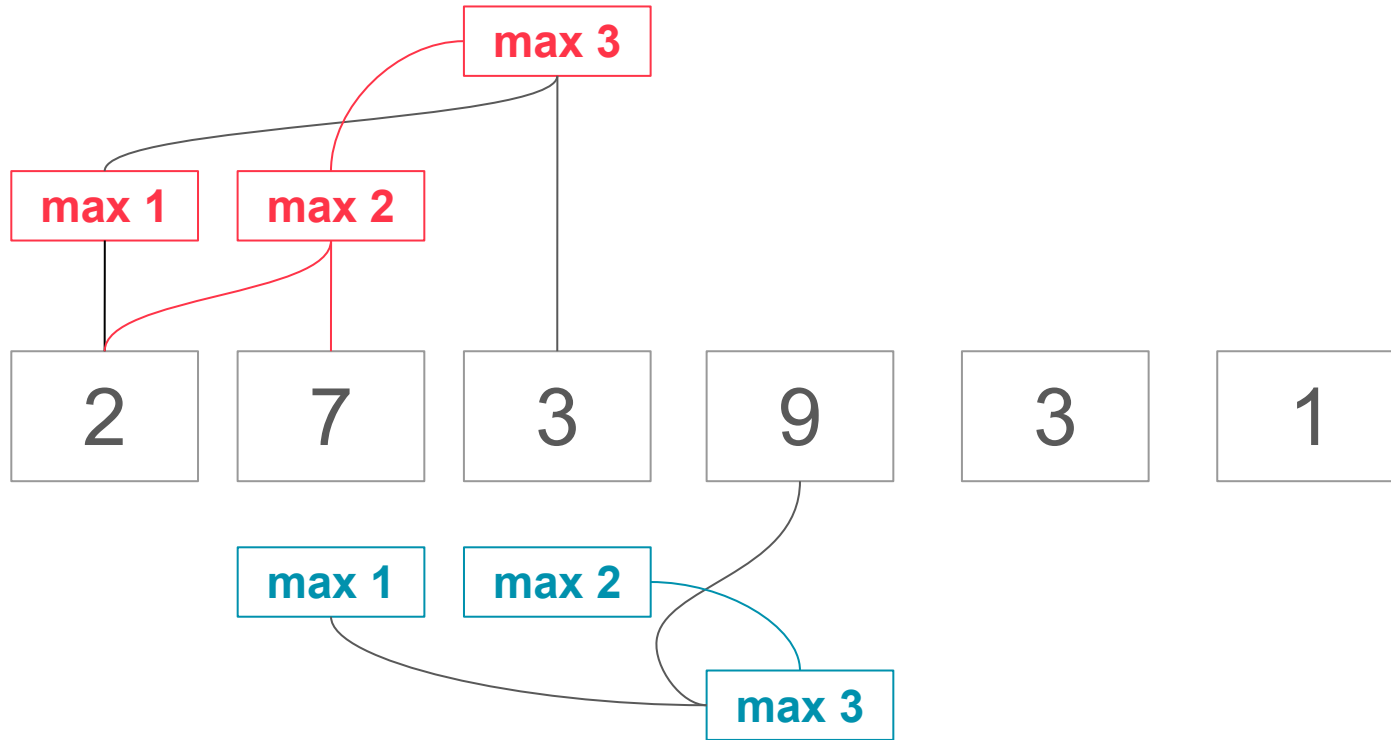


Option A

Exploring the special test case



Bottom-Up Dynamic Programming Approach



Challenge: Climbing Stairs

Common Patterns for Dynamic Programming

Common patterns for dynamic programming

1. Fibonacci Sequence
2. Zero-One Knapsack
3. Unbounded Knapsack
4. Egg-Dropping Variation (Applied)
5. Longest Common Substring/Subsequence
6. DP in Grid (Floyd-Warshall Shortest Path)
7. DP in Graphs & Trees (Applied)

Additionally, we'll introduce multi-layered constraints for more robustness

Zero-One Knapsack DP Pattern

RECAP: The fractional knapsack problem

What is the maximum profit that one can make from a knapsack of maximum weight **12 (Could be any number)**? Items can be divided/fractionalised to fit the knapsack.

items	A	B	C	D
weights	1	3	5	7
profits	2	4	7	10

Use the **Greedy approach**. Prioritise based on **profits per weight**.

profits/weights	2	1.333	1.4	1.428
-----------------	---	-------	-----	-------

How we solved the fractional knapsack (W=12)

items	A	B	C	D
weights	1	3	5	7
profits	2	4	7	10
profits/weights	2	1.333	1.4	1.428

- **$W = 12$** ; choose item with highest profit i.e. A: **$P = 2$**
- **$W = 12 - 1 = 11$** ; choose next valuable item i.e. D: **$P = 2 + 10 = 12$**
- **$W = 11 - 7 = 4$** ; choose next valuable item i.e. C:
 - C cannot fit; we fractionalise it to fit i.e. $\frac{4}{5} (W/w(c))$
 - Thus, the profit will be: $12 + (\frac{4}{5} * 7) = 17.6$

What is the 0-1 knapsack problem?

What is the maximum profit that one can make from a knapsack of maximum weight **8 (Could be any number)**? Items cannot be divided, an item is either included in the knapsack or not.

items	A	B	C	D
weights	1	3	5	7
profits	2	4	7	10

Why we can't use the greedy approach?

- Maximum Weight = 8
- Items = X,Y,Z
- Weights = 3,5,7
- Profits = 4,7,10
- Profits/Weight = 1.33,1.4,1.43

The Greedy Approach:

- Choose the valuable item – Z
- Rem **weight of 1**, no other item can fit in the knapsack
- Max **profit of 10**

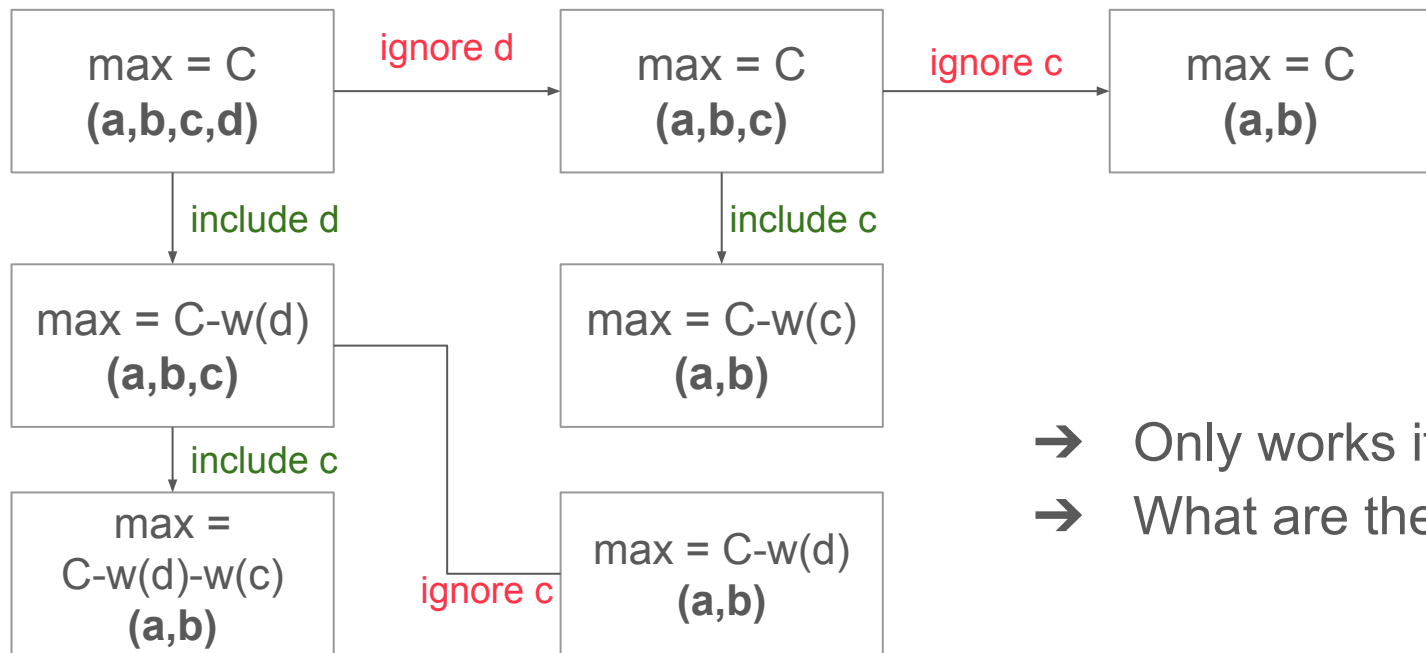
The global optimal solution:

- Choose X and Y
- Max profit of **4 + 7 = 11**

Top-Down Approach

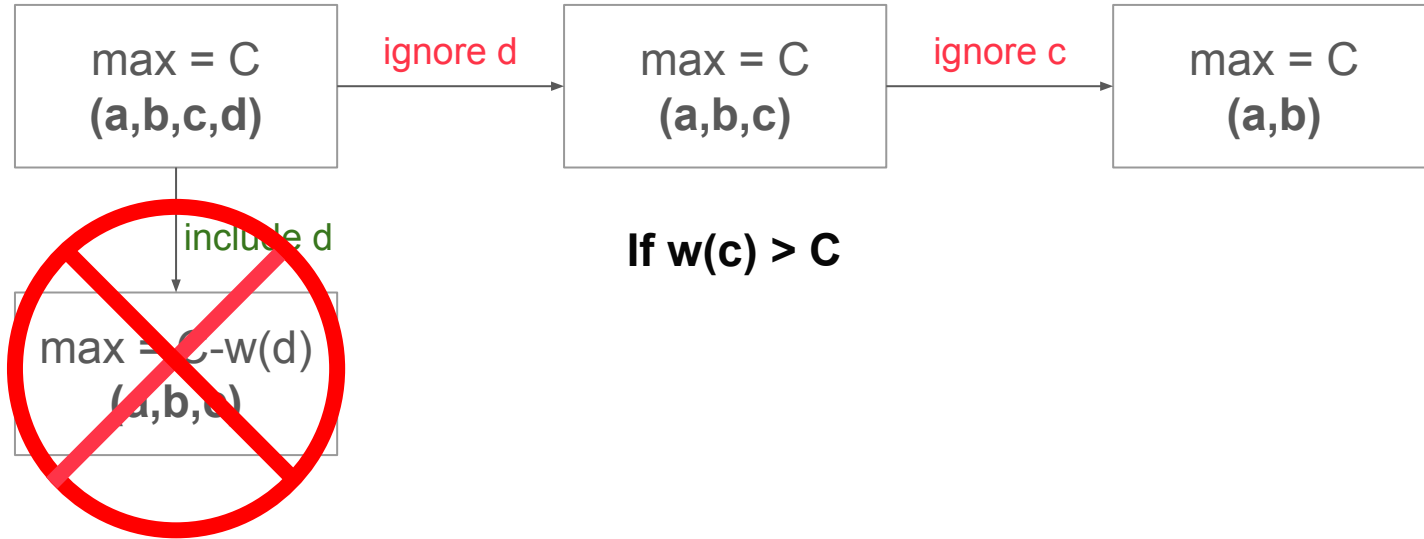
Find the recursive subproblem (Top-Down)

items = a,b,c,d | weights = 1,3,5,7 | profits = 2,4,7,10 | Max Weight is C



- Only works if $w(i) \leq C$
- What are the base cases?

If item weighs more than the available weight?



If $w(c) > C$

We cannot include
d; if $w(d) > C$

What are the base cases?

- Base 1: If we only have one item to choose from:
 - ◆ Include if it fits the knapsack ($w(a) \leq C$)
 - ◆ Ignore it if it does not fit the knapsack ($w(a) > C$)
- Base 2: If we have no items to choose, return 0

Bottom-up Approach

Bottom-up approach

Weights = 1,3,5,7 (n) | Profits = 2,4,7,10 | Maximum Weight of 8 (m)

Maximum Weights

	0	1	2	3	4	5	6	7	8
w=0 p=0	0	0	0	0	0	0	0	0	0
w=1 p=2	0	2	2	2	2	2	2	2	2
w=3 p=4	0	2	2	4	6	6	6	6	6
w=5 p=7	0	2	2	4	6	7	9	9	11
w=7 p=10	0	2	2	4	6	7	9	10	12

Big-O Notation for 0-1 Knapsack

→ Normal Recursion:

- ◆ Runtime complexity: $O(2^n)$
- ◆ Space complexity: $O(1)$

→ Bottom-Up Dynamic Programming:

- ◆ N is the weight of knapsack, M is the number of items
- ◆ Runtime complexity: $O(n*m)$
- ◆ Space complexity: $O(m)$

→ Top-Down Dynamic Programming:

- ◆ Runtime complexity: $O(n*m)$
- ◆ Space complexity: $O(n*m)$

Challenge: Target Sum 0-1 Knapsack

Unbounded Knapsack DP Pattern

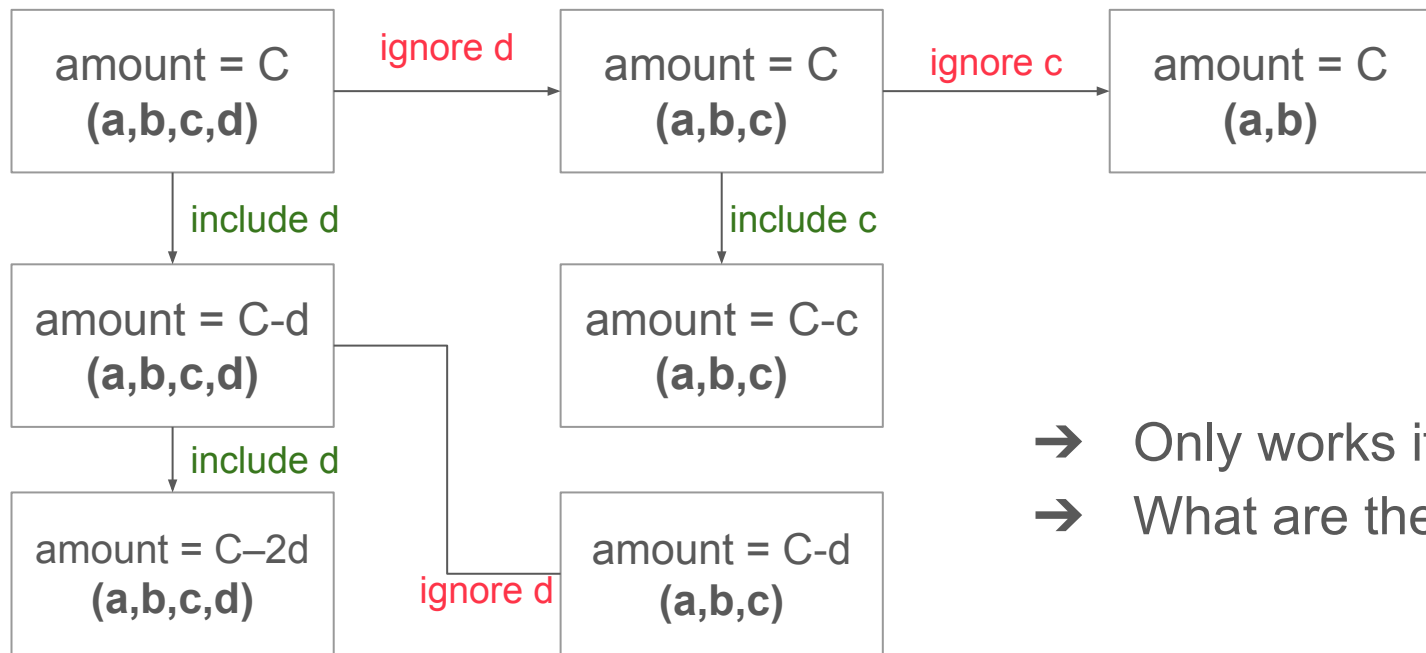
Introducing the Unbounded Knapsack Problem

You have an unlimited types of coins of different denominations: **1,3,4,5**.
What is the minimum number of coins you can use to issue **change of amount 7 (This could be any amount A)**?

You have an unlimited supply of every coin denomination

Find the recursive subproblem (Top-Down)

coins = a,b,c,d | Target change is C



- Only works if coin(i) ≤ C
- What are the base cases?

Bottom-Up Approach

Coins = 1,3,4,5 (Unlimited Amount) | Target Amount = 7

Amount	0	1	2	3	4	5	6	7
Min coins	0							
Operations								

Challenge: Rod Cutting Challenge

Longest Common Subsequence DP Pattern (LCS)

Understanding Substrings & Subsequences

Subsequence: “abcdefghij” abd, abef adg (Order, what follows what)

Substring: “abcdefghij” – abcd, cde, fgh (Order + Continuity)

This pattern applies to:

1. Longest common subsequence
2. Longest common substring
3. Longest palindromic substring aba abba
4. Longest palindromic subsequence $\text{LCS}(\text{str}, \text{reverse}(\text{str}))$
5. Longest common prefix/suffix (Creative patterns)

Top-Down Dynamic Programming

- Text1 = “abcdefghi” | Text2 = “cdgi” (4)
- Text1 = “abcdefghi” | Text2 = “ecdgi”
- Find the recursive sub-problem
- Add memoization to the top-down approach

Bottom-up Dynamic Programming

If last letters are same: diagonal + 1 | max(left,top)

	0	a	b	c	d	e	f	g	i
0	0	0	0	0	0	0	0	0	0
e	0	0	0	0	0	1	1	1	1
c	0	0	0	1	1	1	1	1	1
d	0	0	0	1	2	2	2	2	2
g	0	0	0	1	2	2	2	3	3
i	0	0	0	1	2	2	2	3	4

Longest Palindromic Subsequence

Palindrome: reverse = word

Aba = aba | Cyril = liryC | Arsenal = lasenra | Ana = ana | Madam =
madam | 1001 = 1001 | 123 = 321 |

Abdmefgadmnamop = madam

Hint: text1 = abdmefgadmnamop | text2 = pomandagfemdba

Egg-Dropping DP Pattern (Final Capstone)

Applied Dynamic Programming

Where is DP commonly used?

1. **Constrained optimisation problems:** Knapsack problems
 - a. Maximise profit (Resource allocation, optimal portfolios e.g. FPL, Stock)
 - b. Minimise costs (Shortest path, Matrix Multiplication)
2. Seam Carving (Image Processing)
3. Sequence Alignment (ML and Genomics)
4. Git Merge (Longest Common Subsequence)
5. Dynamic-Key Cryptography (Keys that change over time)